

32 — GENETIC PROGRAMMING FOR SYMBOLIC REGRESSION

Aim

To evolve a symbolic mathematical expression that closely fits a given dataset using Genetic Programming.

Software Required

- Python 3.x
- Jupyter Notebook / Google Colab
- NumPy
- gplearn

Installation

```
!pip install gplearn
```

Procedure

1. Generate input values using NumPy.
2. Generate target values using a mathematical function.
3. Create a SymbolicRegressor.
4. Generate an initial population of mathematical expressions.
5. Evaluate the fitness of each expression using prediction error.
6. Select better expressions.
7. Apply crossover and mutation.
8. Repeat for several generations.
9. Select the best evolved expression.
10. Display the evolved expression and R^2 score.

Code

```
import numpy as np

from gplearn.genetic import SymbolicRegressor

X = np.linspace(-5, 5, 50).reshape(-1, 1)

y = (X**2 + 3*X + 2).ravel()

model = SymbolicRegressor(
```

```
population_size=500,
generations=20,
function_set=('add', 'sub', 'mul'),
random_state=1
)
model.fit(X, y)
print("Evolved program:", model._program)
print("R^2 score:", model.score(X, y))
```

Output:

```
print("Evolved program:", model._program)
print("R^2 score:", model.score(X, y))

... Evolved program: add(sub(0.941, -0.952), sub(add(add(mul(-0.003, -0.729), sub(add(X0, 0.357), mul(0.358, 0.500))), sub(add(X0, -0.301), sub(
R^2 score: 0.9999999994162309
```
